1. Variables and Data Types

Definition:

A variable is a named storage location that holds a value, which can change during program execution. JavaScript provides three ways to declare variables:

1. `var` - The old way; function-scoped.
2. `let` - The modern way; block-scoped.
3. `const` - A constant; cannot be reassigned.

Syntax:

javascript Copy Edit

```javascript
var x = 10; // Function-scoped variable let y = 20; // Block-scoped variable const z = 30; // Constant variable, cannot be changed
```

Example from Your File:

javascript Copy Edit

```javascript
let x = 10; let y = 20; let z = x + y;
```

Explanation:

- `x` and `y` are assigned values `10` and `20`, respectively.
- `z` stores the sum of `x` and `y`, which is `30`.
- The `let` keyword ensures that these variables are **only accessible within the block they are declared in**, preventing unintended modifications.

---

2. Arithmetic Operations

Definition:

Arithmetic operations perform mathematical calculations such as addition, subtraction, multiplication, and division.

Operators:

Operator	Description	Example
`+`	Addition	`let sum = a + b;`
`-`	Subtraction	`let diff = a - b;`
`*`	Multiplication	`let product = a * b;`
`/`	Division	`let quotient = a / b;`
`%`	Modulus (remainder)	`let remainder = a % b;`
`++`	Increment	`a++` (adds 1)
`--`	Decrement	`a--` (subtracts 1)

Example from Your File:

javascript Copy Edit

```javascript
let a = 3; x = a * 150; x += 10;
```

Explanation:

- `x = a * 150` multiplies `3` by `150`, storing `450` in `x`.
- `x += 10` increases `x` by `10`, making it `460`.
- This shorthand notation (`+=`) is equivalent to `x = x + 10`, improving readability.

---

3. String Concatenation

Definition:

String concatenation combines multiple strings using the `+` operator.

Syntax:

javascript Copy Edit

```
let fullString = "Hello" + " " + "World!";
```

Example from Your File:

javascript Copy Edit

```
let str1 = "hello world"; let str2 = ", this is Jason";
document.getElementById("demo").innerHTML = str1 + str2;
```

Explanation:

- `str1 + str2` joins `"hello world"` with `", this is Jason"`, resulting in `"hello world, this is Jason"`.
- Concatenation is often used when generating dynamic HTML content.

---

4. Type Comparison

Definition:

JavaScript allows two types of comparisons:

1. **Loose Comparison (`==`) -** Converts data types before comparing.
2. **Strict Comparison (`===`) -** Checks both **value and data type** without conversion.

Example from Your File:

javascript Copy Edit

```
let num1 = "5"; // String let num2 = 5; // Number
document.getElementById("demo").innerHTML = (num1 === num2);
```

Explanation:

- `"5" == 5` returns `true` because JavaScript **converts** `"5"` into a number before comparison.
- `"5" === 5` returns `false` because a **string** is not equal to a **number**.
- Using `===` is recommended to avoid unintended behavior.

5. Logical Operations

Definition:

Logical operators allow conditions to be combined:

- `&&` (AND) → `true` only if **both** conditions are `true`.
- `||` (OR) → `true` if **at least one** condition is `true`.
- `!` (NOT) → Reverses the truth value.

Example from Your File:

javascript Copy Edit

```javascript
document.getElementById("demo").innerHTML = (x < 10 && y > 1);
document.getElementById("demo").innerHTML = !(x === y);
```

Explanation:

- `(x < 10 && y > 1)` returns `true` **only if** `x < 10` and `y > 1`.
- `!(x === y)` negates the result of `x === y`.

---

6. Objects and Accessing Properties

Definition:

An **object** is a collection of key-value pairs, used to store related data.

Syntax:

javascript Copy Edit

```javascript
let objectName = {key1: value1, key2: value2};
```

Example from Your File:

javascript Copy Edit

```
let obj = {name: "abc", age: 20, class: 10}; document.getElementById("demo").innerHTML = obj.class;
```

Explanation:

- `obj.class` retrieves the value `10` from the object.
- Objects help organize and group related data efficiently.

---

7. Conditional Statements

Definition:

Conditional statements allow code execution based on conditions.

Syntax:

javascript Copy Edit

```
if (condition) { // Code executes if condition is true } else if (anotherCondition) { // Code executes if anotherCondition is true } else { // Code executes if no conditions are met }
```

Example from Your File:

javascript Copy Edit

```
let time = new Date().getHours(); let greeting; if (time < 10) { greeting = "Good morning"; } else if (time < 20) { greeting = "Good day"; } else { greeting = "Good evening"; }
```

Explanation:

- The script checks the current hour (`getHours()`) and assigns the appropriate greeting.
- If the hour is **before 10 AM**, `"Good morning"` is displayed.
- If the hour is **between 10 AM and 8 PM**, `"Good day"` is displayed.

- Otherwise, `"Good evening"` is displayed.

---

8. Switch Statement

Definition:

A `switch` statement selects one of many code blocks to execute based on a value.

Syntax:

javascript Copy Edit

```javascript
switch (expression) { case value1: // Code break; case value2: // Code break; default: // Code }
```

Example from Your File:

javascript Copy Edit

```javascript
let day; switch (new Date().getDay()) { case 0: day = "Sunday"; break; case 1: day = "Monday"; break; default: day = "Invalid day"; }
```

Explanation:

- `getDay()` returns the day as a number (0 for Sunday, 1 for Monday, etc.).
- The `switch` statement matches the number and assigns the correct day name.

Final Thoughts

This JavaScript file covers **essential concepts**, including: ✅ **Variables and Data Types**
✅ **Arithmetic and String Operations**
✅ **Logical and Comparison Operators**
✅ **Objects and Arrays**
✅ **Conditional and Switch Statements**